

Лабораторна робота №4 Застосування патернів програмування

Виконала Філіпова Єлизавета 301 група

1. Аналізуючи діаграми класів (Лабораторна робота №3), обґрунтувати застосування підібраних для реалізації патернів.

На основі аналізу структури класів інформаційної системи мовної школи (User, Student, Teacher, Group, Payment тощо) було обрано та обґрунтовано **чотири шаблони проектування**, що належать до різних категорій (породжувальні, структурні та поведінкові).

1. Патерн Singleton -Породжувальний шаблон

1. **Головна ідея патерну:** Гарантувати, що у всій програмі існує лише один єдиний екземпляр певного класу, і надати до нього одну спільну точку доступу.
2. **Чому це критично для мовної школи:** У системі є спільний ресурс - база даних CourseSystem_DB. Коли додаток запускається, модулі авторизації (AuthService), навчання (EduManager) та оплат (BillingService) починають одночасно слати туди запити. Якщо кожна функція при кожному кліку студента створюватиме новий об'єкт з'єднання з базою, сервер миттєво заблокує підключення через вичерпання ліміту. Крім того, якщо два адміністратори одночасно спробують відредагувати розклад однієї групи через різні об'єкти підключення, дані в базі можуть просто зіпсуватися (виникне стан гонитви даних).
3. **Як саме це влаштовано всередині коду (механіка):**
 - a. Ховається конструктор класу роботи з БД, роблячи його приватним (private DatabaseConnection()). Тепер ніхто в програмі не зможе написати new DatabaseConnection().
 - b. Всередині самого класу створюється закрите статичне поле instance, де зберігатиметься той самий єдиний об'єкт.
 - c. Створюється публічний статичний метод getInstance(). Коли будь-який модуль програми викликає цей метод, клас перевіряє: якщо об'єкт ще не створено -він його створює. Якщо об'єкт уже є в пам'яті - клас просто повертає посилання на нього.
4. **Результат :** Існує повний контроль над доступом до бази даних. Пам'ять сервера використовується максимально економно, а запити виконуються в чіткій черзі без конфліктів.

2. Патерн Facade - Структурний шаблон

- **Головна ідея патерну:** Створити один простий "зовнішній" клас, який ховає за собою всю складність, заплутаність та велику кількість методів інших "внутрішніх" класів системи.
- **Чому це критично для мовної школи:** У Лабораторній роботі №3 я розділила логіку сервера на три незалежні сервіси: AuthService.dll, EduManager.dll та BillingService.dll. Щоб просто записати нового студента на навчання, клієнтському додатку (LanguageSchool.exe) довелося б зробити купу

кроків: спочатку звернутися до сервісу авторизації та створити профіль, потім зайти в сервіс навчання, знайти потрібну групу, перевірити запис, записати туди людину, а після цього звернутися до фінансового сервісу для генерації інвойсу. Це створює проблему. Якщо хоч трохи змінюється логіка всередині фінансового модуля, доведеться повністю переписувати ткод клієнтського додатка.

- **Як саме це влаштовано всередині коду :** Створюється загальний клас-контролер . Цей клас знає, як працюють усі три підсистеми всередині сервера. Для клієнтського додатка він відкриває лише кілька простих, зрозумілих методів, наприклад: `registerNewStudent(Data)`. Клієнтська програма просто викликає цей один метод і чекає на результат. А вже сам "Фасад" всередині сервера по черзі викликає складні методи реєстрації, перевірки груп та виставлення рахунків.
- **Результат для :** Клієнтська програма стає дуже простою. Вона нічого не знає про внутрішні особливості сервера. Є можливість повністю змінювати, оновлювати чи оптимізувати серверний код, але інтерфейс взаємодії для користувача залишиться стабільним і нічого не зламається.

3. Патерн Strategy - Поведінковий шаблон

- **Головна ідея патерну:** Виділити сімейство схожих алгоритмів в окремі класи і зробити їх взаємозамінними прямо під час роботи програми.
- **Чому це критично для мовної школи:** Клас `Payment` (Оплата) має вміти проводити грошові транзакції. Проте студенти нашої школи платять абсолютно різними способами: хтось використовує міжнародну картку , хтось платить оналайн через різні платформи, а хтось списує накопичені внутрішні бонуси чи використовує промокоди на знижку. Якщо писати код стандартним шляхом, метод `processPayment()` перетвориться на гігантське дерево з умов . Це грубе порушення архітектурних правил. Щоразу, коли керівництво школи вирішить підключити нову платіжну систему (наприклад, `PayPal` або `Apple Pay`), доведеться відкривати цей фінансовий клас і змінювати його, ризикуючи випадково зламати вже налаштований та протестований механізм оплат.
- **Як саме це влаштовано всередині коду :**
 1. Створюється загальний інтерфейс `IPaymentStrategy`, в якому прописуємо один єдиний абстрактний метод `pay(float amount)`.
 2. Для кожного конкретного способу оплати створюється окремий маленький клас, який реалізує цей інтерфейс (наприклад, `StripeStrategy`, `PayStrategy`, `BonusStrategy`). У кожному з цих класів логіка оплати прописана по-своєму.
 3. Головний клас `Payment` більше не містить жодних умов `if-else`. Він просто має всередині змінну типу `IPaymentStrategy`. Коли студент на екрані телефону обирає, наприклад, "Оплата картою", програма динамічно підставляє об'єкт `StripeStrategy` у наш клас оплат.
- **Результат:** Фінансова система стає максимально гнучкою. Можна додавати хоч по 10 нових платіжних систем на місяць — для цього достатньо просто створити новий файл з окремим класом, взагалі не чіпаючи і не переписуючи старий робочий код.

4. Патерн Observer - Поведінковий шаблон

- **Головна ідея патерну:** Створити таку систему зв'язків, щоб при зміні стану одного об'єкта всі інші об'єкти, які від нього залежать, автоматично отримували сповіщення про це.
- **Чому це критично для мовної школи:** У системі є об'єкти, стан яких постійно міняється адміністраторами. Наприклад, об'єкт `Group` (Група) змінює статус із "Набір" на "Активна", коли набралось 10 студентів. Або в об'єкті `Schedule` (Розклад) адміністратор переносить заняття на іншу годину чи міняє клас. Зараховані студенти (`Student`) та закріплений вчитель (`Teacher`) повинні

миттєво про це дізнатися. Якщо реалізувати це шляхом опитування -коли додаток кожного студента кожні 5 секунд шле запит на сервер, це може призвести до того що він “впаде”

- **Як саме це влаштовано всередині коду :**

1. Створюється інтерфейс Observer (Спостерігач) із методом update(string message). Класи Student та Teacher реалізують цей інтерфейс, тобто вони автоматично стають "спостерігачами".
2. Об'єкт Group (або Schedule) стає "Суб'єктом" (Subject). Всередині нього створюється динамічний список підписників: List<Observer> subscribers. Коли студент записується в групу, програма додає його об'єкт до цього списку.
3. Як тільки адміністратор змінює час уроку в розкладі групи, об'єкт групи автоматично запускає внутрішній цикл по всьому списку subscribers і викликає метод subscribers[i].update().

- **Результат :** Можна повністю позбутися зайвого мережевого трафіку та пустих запитів до бази даних. Студенти та вчителі отримують інформацію в режимі реального часу (як Push-повідомлення) і лише тоді, коли подія дійсно відбулася. При цьому об'єкт групи залишається повністю незалежним від того, як саме влаштований екран додатка у студента.

Висновок: Використання патернів **Singleton**, **Facade**, **Strategy** та **Observer** допомогло вирішити важливі завдання: ми розвантажили базу даних, сховали від користувача складну внутрішню логіку сервера, зробили зручну систему для додавання нових видів оплати та налаштували швидкі сповіщення про зміну розкладу.

2) Використовуючи лабораторні роботи із дисциплін професійної підготовки, створити репозиторій, розмістити файли проектів. Надати доступ викладачам на github / gitlab ресурсі.

 [GitHub - lizafilipova/language-courses](https://github.com/lizafilipova/language-courses)